



TRABAJO FINAL INTEGRADOR

PROGRAMACIÓN III – 2026 1er cuatrimestre
TECNICATURA UNIVERSITARIA EN DESARROLLO WEB

OBJETIVOS

El Trabajo Final Integrador tiene como objetivos que el estudiante:

- Integrar y aplicar los contenidos de programación del lado del servidor en el desarrollo de un sistema funcional.
- Diseñar e implementar servicios web basados en arquitecturas REST.
- Utilizar estructuras de datos apropiadas para la gestión eficiente de la información.
- Incorporar mecanismos de comunicación en tiempo real en aplicaciones cliente-servidor.

CONDICIONES DE ENTREGA

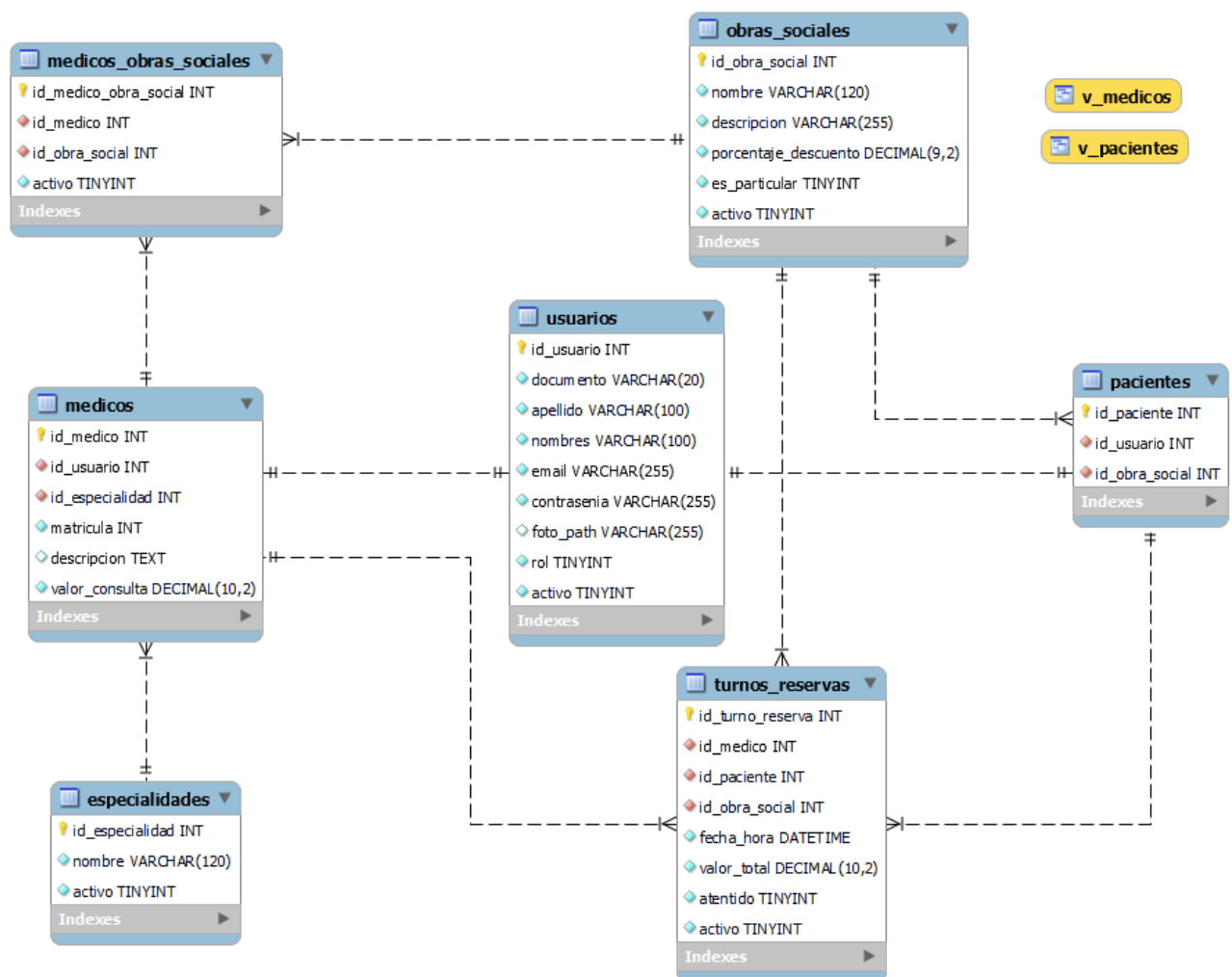
- El Trabajo Final Integrador deberá ser:
 - Realizado en forma grupal.
 - Los grupos deberán contar con un mínimo de 5 y un máximo de 6 alumnos.
 - Cargado en la sección del Campus Virtual como un archivo de texto plano (txt) que cuente con la URL al repositorio GitHub. Además, para asegurar el acceso del equipo docente, se sugiere cargar un archivo comprimido en formato ZIP, RAR, TAR.GZ u otro formato similar. con el proyecto completo excluyendo las dependencias (node_modules).
 - Deberá indicarse nombre de grupo, apellido y nombre de los integrantes.
 - Entregado antes de la fecha límite informada en el campus.
- Las soluciones deben ser de autoría propia. Aquellas que se detecten como idénticas entre diferentes grupos serán desaprobadas para todos los involucrados.
- Se valorarán la exactitud, eficiencia y prolijidad (identación y otras buenas prácticas) de las soluciones planteadas, así como también la calidad de la exposición realizada.
- Los integrantes del equipo deberán realizar un video de exposición del proyecto, detallando su funcionamiento y las decisiones de diseño tomadas. Este video deberá:
 - Mostrar la pantalla del dispositivo donde se visualice el código y la aplicación en ejecución.
 - Exponer la voz de todos los integrantes explicando el proyecto.
 - Idealmente, capturar en video a los integrantes mientras realizan la explicación.
 - Cargarse en YouTube, Google Drive, Vimeo u otro.

FECHAS DE ENTREGA

- Primera entrega (avance funcional mínimo): **08/05/2026**
 - BREAD (Browse, Read, Edit, Add, Delete) completo de alguna entidad del API contemplando las mejores prácticas vistas en clase.
- Entrega final (versión completa): **16/06/2026**
 - Con todos los requerimientos completados.
- Recuperatorio: **29/06/2026**

ENUNCIADO

La clínica médica para la que desarrolló el sitio web y aplicación con funcionalidades del lado del cliente desea completar el proyecto para lo cual le encarga el desarrollo de una aplicación del lado del servidor que permita registrar el siguiente modelo de datos:



REQUISITOS FUNCIONALES

Según los roles de la aplicación se listan las funcionalidades que deben estar permitidas:

Médico (ROL = 1)

- Iniciar sesión.
- Listar turnos propios.
- Marcar un turno como atendido.

Paciente (ROL = 2)

- Iniciar sesión.
- Crear reservas (turnos propios).
- Listar turnos propios.
- Listar especialidades.
- Listar todos los médicos y de una especialidad.

Administrador (ROL = 3)

- Iniciar sesión.
- Listar, crear y editar especialidades.
- Asociar médicos con especialidades.
- Listar, crear y editar obras sociales.
- Asociar médicos con obras sociales.
- Asociar pacientes con obras sociales.
- Registrar un turno para un paciente, médico y fecha.
- Obtener estadísticas de atenciones.

RESTRICCIONES Y REGLAS DE NEGOCIO:

- Las estadísticas deben generarse exclusivamente mediante procedimientos almacenados (stored procedures).
- Los informes en PDF deben contener información sobre los turnos: cantidad, pacientes, obras sociales, etc.
- El campo **valor_total** de la tabla **turnos_reservas** se calcula de la siguiente manera:
medicos.valor_consulta - (obras_sociales.porcentaje_descuento * medicos.valor_consulta) para los casos en que la obra social **NO** sea particular (**es_particular = 0**). Si la obra social **ES** particular (**es_particular = 1**)
valor_total = medicos.valor_consulta.
- Los “delete” no serán borrados físicos, se utilizarán “soft delete”, es decir se utilizará el campo **activo** para indicar si el registro de la tabla está borrado o no. En este sentido para ser consistente todos los registros que se busquen de la base de datos deberán cumplir el criterio **activo = 1**.

ASPECTOS TÉCNICOS REQUERIDOS

- Autenticación con JWT.
- Autorización por roles.
- Uso del framework Express.
- Persistencia de datos en MySQL.
- Utilización de Transacciones MySQL.
- Buen manejo de errores y respuestas HTTP apropiadas.
- Documentación del API haciendo uso de Swagger.
- Utilización de variables de entorno para el manejo de información sensible.
- CORS para habilitar / deshabilitar futuras conexiones desde el cliente web (front-end).
- Middlewares:
 - Validaciones. Ej.: express-validator.
 - Registro de solicitudes. Ej.: Morgan.
 - Carga de archivos. Ej.: Multer.

EXTRAS

Cada grupo podrá agregar una funcionalidad extra al desarrollo.

Lista de ejemplos:

- Permitir al profesional agregar comentarios u observaciones respecto de la atención.
- Sistema de auditoría: historial de acciones por usuario.
- Registro de usuarios: pacientes, médicos y administradores.
- Reinicio de contraseña para los usuarios.
- Incluir funcionalidades SSE o WebSockets que permitan saber a los pacientes cuál es el próximo turno a llamar.
- Endpoint con LLM para identificar posibles diagnósticos.

RÚBRICAS

Escala por criterio:

- 0 = No cumple / incorrecto
- 1 = Parcial / incompleto / con errores
- 2 = Cumple correctamente y de forma consistente

Nota = Puntaje total / 10

1. CRUD de entidades

- 0: Faltan operaciones básicas o no funcionan.
- 1: CRUD incompleto o con errores de persistencia.
- 2: CRUD completo y funcional para las entidades requeridas.

2. Reglas de negocio del dominio

(Ej.: disponibilidad de turnos, reservas válidas, restricciones)

- 0: No implementadas o incorrectas.
- 1: Parcialmente implementadas o inconsistentes.
- 2: Correctas y coherentes en toda la API.

3. Seguridad y roles (JWT)

- 0: Sin autenticación o mal implementada.
- 1: JWT funcional pero sin control adecuado de roles.
- 2: Autenticación y autorización por rol correctamente aplicada.

4. Validaciones de datos (express-validator o similar)

- 0: Sin validaciones o acepta datos inválidos.
- 1: Validaciones incompletas o inconsistentes.
- 2: Validaciones completas y correctas en todas las entradas.

6. Endpoints y diseño REST

- 0: Rutas incorrectas o no REST.
- 1: REST parcial o inconsistente.
- 2: Diseño REST claro (recursos, verbos, códigos HTTP).

7. Persistencia y transacciones

- 0: Errores de guardado o inconsistencias.
- 1: Persistencia funcional sin manejo de casos críticos.
- 2: Persistencia correcta con manejo adecuado de transacciones.

8. Funcionalidades técnicas requeridas (SP, PDF, archivos, etc.)

- 0: No implementadas.
- 1: Implementadas parcialmente o con errores.
- 2: Implementadas correctamente y funcionales.

9. Documentación y uso (Swagger / README)

- 0: Sin documentación.
- 1: Documentación incompleta o poco clara.
- 2: API documentada y utilizable (Swagger o similar).

10. Defensa / explicación técnica (Video o exposición)

- 0: No puede explicar decisiones o código.
- 1: Explicación superficial o con dudas.
- 2: Explica con claridad arquitectura, decisiones y funcionamiento.